



Accenture Duck Creek

EXAMPLE Forms™

Reference Guide





Copyright © 2000-2011 Duck Creek Technologies, Inc. All rights reserved.

The software contains proprietary information of Duck Creek Technologies. It is provided under a license agreement containing restrictions on use and disclosure and is also protected by copyright law. Reverse engineering of the software is prohibited.

Due to continued product development, this information may change without notice. The information and intellectual property contained herein is confidential between Duck Creek Technologies and the client and remains the exclusive property of Duck Creek Technologies. If you find any problems in the documentation, please report them to us in writing. Duck Creek Technologies does not warrant that this document is error-free.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording or otherwise—without the prior written permission of Duck Creek Technologies, Inc.

Duck Creek Technologies. <http://www.duckcreektech.com>

EXAMPLE Forms™ Reference Guide

This document contains information about the features and functions of EXAMPLE Forms™ and instructions for creating, generating, and managing forms.

This document also contains two appendixes. *Commercial Lines Forms* addresses special information concerning Commercial Lines forms, including forms standards, an explanation of forms inheritance, and special considerations for forms. *EXAMPLE Forms™ Customization* describes general customization practices for Duck Creek Technologies, Inc. products, including EXAMPLE Forms™.

EXAMPLE Forms™ Functionality

The EXAMPLE Forms™ system has the following important features:

- Three forms levels to meet a wide range of needs
- Support for:
 - Creating custom forms
 - Defining rules for forms
 - Creating print jobs (groups of forms) that can be generated at various points in the policy lifecycle
 - Saving generated forms, e-mailing them, or attaching them to a policy
 - Printing forms using the Duck Creek Technologies, Inc. Form Print Utility
- EXAMPLE Express® features that allow users to interface with policy forms, including:
 - Policy Forms page (For information on this page, see *Policy Forms Page*.)
 - Create Policy Documents page

In addition to the above features, the EXAMPLE Forms™ system has a number of other special features that can help simplify and streamline forms processing. For information about these features, see *Special Functions of the Forms Engine*.

Policy Forms Page

The Policy Forms page in EXAMPLE Express® allows users to interact with the EXAMPLE Forms™ system. On the Policy Forms page, users can:

- View all forms on a given policy and all forms available for that policy (driven by Manuscript rules)
- Select which forms to print
- Add forms to the policy
- Remove forms from the policy (driven by Manuscript rules)
- Preview forms
- Edit form data



When attempting to view forms in IE8 when the pop-up blocker is enabled, ensure that the URL for EXAMPLE Express® has been added to the list of trusted sites in the Internet Properties dialog box (accessed through the Microsoft Windows Control Panel) or to the list of allowed sites in the Pop-up

Blocker Settings dialog box (accessed through the Internet Properties dialog box in the Microsoft Windows Control Panel).

If you have configured a custom action in EXAMPLE Author® to initialize and display print jobs in EXAMPLE Express®, the Policy Forms page in EXAMPLE Express® displays. This page appears as follows:

Policy Forms

Select the forms you wish to print for this transaction. If you print a form that is not already on the policy, it will be added to the policy. If you wish to remove a form from the policy, click Remove Form .

Show: All Forms

Search: Search Clear

[Restore Defaults](#)

☐ Print	On Policy?	Form	Title	Category	Order	Actions
☒	Add	CGDS011001	COMMERCIAL GENERAL LIABILITY DECLARATIONS		1	
☒	Add	CG00011207	COMMERCIAL GENERAL LIABILITY COVERAGE FORM (OCCURRENCE VERSION)		2	
☒	Add	CG00680509	RECORDING AND DISTRIBUTION OF MATERIAL OR INFORMATION IN VIOLATION OF LAW EXCLUSION		3	
☐		CG02241093	EARLIER NOTICE OF CANCELLATION PROVIDED BY US (FOR USE WITH CGL, LIQUOR, POLLUTION AND PRODUCTS POLICIES)		4	 
☒	Add	CG04221185	POLLUTION LIABILITY COVERAGE EXTENSION (FOR USE WITH CGL POLICIES)		5	
☒	Add	CG20070704	ADDITIONAL INSURED - ENGINEERS, ARCHITECTS, OR SURVEYORS		6	
☐		CG21000798	EXCLUSION - ALL HAZARDS IN CONNECTION WITH DESIGNATED PREMISES		7	 
☐		CG21041185	EXCLUSION - PRODUCTS/COMPLETED OPERATIONS HAZARD		8	
☐		CG21160798	EXCLUSION - DESIGNATED PROFESSIONAL SERVICES		9	 
☐		CG21310509	LIMITED EXCLUSION - DESIGNATED OPERATIONS COVERED BY A CONSOLIDATED (WRAP-UP) INSURANCE PROGRAM		10	 

1-10 11-20 21-30 31-40 41-50 51-60 61-67

Prev Page [Next Page](#)

Return to Policy

The list of forms on the Policy Forms page has the following columns.

Column Heading	Description
----------------	-------------

Print	Defines which forms to print for this transaction. Form properties defined in the EXAMPLE Author® Forms Mapper tool determine the default state of the check boxes in this column (<i>selected</i>, <i>unselected</i>, or <i>mandatory</i>). Each check box can be: • Selected — If selected, the form will print. • Unselected — If unselected (the check box is cleared), the form will not print. • Selected and grayed out (mandatory) — If selected and grayed out, the form is mandatory. The form will print, and the user cannot modify the print status. • Unselected and grayed out — If unselected and grayed out, the form cannot be selected for printing. This usually occurs on endorsement print jobs when a form is designated as a "mandatory" form, but no changes have occurred to the form since the last transaction. • Hidden — If a form does not have a corresponding check box, the form cannot be selected for printing. This usually occurs when a form is on the policy but not available for printing on the current transaction (<i>i.e.</i>, the form is not a part of the current print job or the form rule is false). Forms selected to print are added to the policy if they do not already exist on the policy.
On Policy?	Defines which forms are on the policy, which will be added to the policy, and which will be removed. Forms that currently exist on the policy are marked with a check mark . Forms that will be added when the transaction is committed are identified by the word Add. Forms to be removed when the transaction is committed are identified by the word Remove.
Form	Form name as it appears in the Manuscript.
Title	Form title. This title can be determined at runtime if set to do so in form properties in the EXAMPLE Author® Forms Mapper tool.
Category	Category as entered in form properties in EXAMPLE Author® Forms Mapper.
Order	Order in which forms are listed in the Manuscript print job.

Actions	Offers the following possible user actions. • Preview  — Loads the current form as a .pdf file in Adobe Acrobat Reader to preview the form and its current fields. • Edit  — Allows the user to edit input fields displayed on the form by displaying them on a dynamically built page. The fields include the data from the active session, and changes made to these fields are stored directly in the active session. Each field can be restricted by settings in the EXAMPLE Author® Forms Mapper tool as read-only and/or required. The Edit  button does not display if all fields on the form are designated as read-only in EXAMPLE Author® Forms Mapper. Also, the user may define field display for editing by referencing a custom page in Page Designer. For more information, see the <i>Page Set</i> and <i>Page</i> properties on the Form Properties <i>Standard Properties Tab</i>. • Remove  — Marks the form for removal from the policy when the transaction is committed. This option appears only if a form is on the policy and is 1) not a mandatory form, or 2) unavailable to print (<i>i.e.</i>, not a valid form on the current print job).  If a user removes a coverage while in a policy, forms tied to that coverage are not automatically removed. Users must manually remove those forms from the Policy Forms page. In addition, users should use caution when removing forms from the Policy Forms page. In some cases, users may have the ability to remove some forms that should not be removed.
---------	---

The Policy Forms page provides a number of options for navigating and selecting forms.

Hyperlink/ Button	Description
Show	Allows the user to filter the list of forms. Options are: • All Forms — Lists all forms that are on the policy as of the last committed transaction and all forms that are valid for the current print job. • Forms on the Policy — Lists only the forms that are on the policy as of the last committed transaction. • Available Forms — Lists only the forms that are valid for the current print job.
Search	Allows the user to search the list of forms by form name or title.  Search criteria on the Policy Forms page is case-sensitive.
Restore Defaults	Restores print selections to their default state. (Removes user selections.)
Prev Page Next Page	Navigates through the list of forms.

Refresh	Re-runs all form rules and refreshes the list of forms (and all form data), preserving any selections the user has made. This option appears only if the <i>PrintJobAutoRefresh</i> configuration setting in the Express\Web.config file is set to 0. If the <i>PrintJobAutoRefresh</i> configuration setting is set to 1, the system automatically performs this function each time the user visits the Policy Forms page.
Return to Policy	Returns the user to the current active quote or policy, saving all selections made.

Special Functions of the Forms Engine

The EXAMPLE Forms™ engine supports the following special functions that help simplify and streamline forms processing:

- Endorsement printing
- Editable form fields
- Form previews (single forms)
- Form printing (single forms)
- Asynchronous processing
- *PublicRecalc* setting

Endorsement Printing

The EXAMPLE Forms™ engine can support the ability to mark for printing only the forms in a policy's forms list that have been added or changed since the last transaction. This function is supported through the following two request attributes:

- The *preserveHistory* attribute on the <ManuScript.printFormsRq> request and the <FormsEngine.processPrintJobRq> request indicates whether to store a history of the forms that are printed. When this attribute is set to 1, the system stores the form list XML to the associated history record for the policy. For the <ManuScript.printFormsRq> request, this attribute defaults to 0. For the <FormsEngine.processPrintJobRq> request, it defaults to 1.
- The *endorsement* attribute on the <FormsEngine.initPrintJobRq> request indicates whether to limit the forms selected to print to added or changed forms only. When the *endorsement* attribute is set to 1, the system performs the following actions:
 - a. Generates the Level 2 forms information.
 - b. Compares the XML of each form to the stored XML for that form.
 - c. If no differences exist for a particular form, causes that form to appear as unselected in the list of forms to be printed.

By default, the *endorsement* attribute is set to 1.

For endorsement printing to be enabled, both the *preserveHistory* attribute and the *endorsement* attribute must be set to 1.



This feature requires EXAMPLE TransACT®. For more information on EXAMPLE TransACT®, see *EXAMPLE TransACT® 2.1 Out-of-the-Box Functionality*.

Editable Form Fields

The EXAMPLE Forms™ system supports the ability to edit fields on a form in EXAMPLE Express®. This can be achieved in two ways:

- Using a Manuscript page created in EXAMPLE Author® Page Designer
- Using a generic EXAMPLE Express® system page

If a PageSet and Page are defined for a form in Forms Mapper, any editing of fields on that form will occur on the specified page. This process uses the `<Manuscript.getPageRq>` request. If there is no specified PageSet and Page, the system produces an auto-generated page using the `<Manuscript.getPageUsingMergeFieldsRq>` request. This request returns a standard `<Manuscript.getPageRs>` response based on information about the form.



Use caution when allowing users to edit Manuscript fields through a form. The Manuscript must be responsible for validating each of the fields to ensure a correct rating solution.

Form Previews (Single Forms)

The EXAMPLE Forms™ engine supports the ability to preview a single form using the `<Manuscript.printFormsRq>` request. To preview a single form, you can specify a specific form name for the *printJob* attribute of the request. To successfully view the form, you must specify a target in the *outputFile* attribute that both EXAMPLE Express® and EXAMPLE Server® can access. EXAMPLE Server® will then write the .pdf file to the specified output file, and the user can view the generated .pdf document in a browser application.



For information about the `<Manuscript.printFormsRq>` and its attributes, see the *EXAMPLE Server® Object Reference*.

Form Printing (Single Forms)

The EXAMPLE Forms™ engine supports the ability to print a single form using the `<FormsEngine.initPrintJobRq>` request. To print a single form, you can specify a specific form name for the *printJob* attribute of the request. Single-form printing can be useful for generating a single file (rather than an entire print job) that you can attach to an e-mail message, save to a folder, or attach to a policy. Single-form printing can also be useful for supporting a preview of the rendered form within EXAMPLE Express® or for delivering the form to a client through the browser.

Asynchronous Processing

Because the `<FormsEngine.initPrintJobRq>` request can take a substantial amount of time, the EXAMPLE Forms™ engine supports the ability to build this request asynchronously. Executing this request asynchronously allows users to perform other actions in the system while EXAMPLE Server® builds a response to the request. It can also help prevent timeout errors that may result from an especially long-running request. To execute the `<FormsEngine.initPrintJobRq>` request as an asynchronous call, you simply define the request with the *async* attribute set to 1.

The response for an asynchronous request will initially return a semaphore ID for the request. This ID allows EXAMPLE Express® to periodically ping EXAMPLE Server®—either over time or through user button press—to check for the completed request. During this process, EXAMPLE Express® opens and displays a loading page.

Once the request is complete, EXAMPLE Server® performs two actions:

- Synchronizes the session in EXAMPLE Express® with the cloned session that EXAMPLE Server® created and used to process the request.

- Retrieves the list of forms from the cloned session using the <Session.getDocumentRq> request for the <formsData> element or the <FormsEngine.listPrintJobRq> request.

(Synchronizing the session and cloned session is necessary because the session may have updated values that were processed by the rules in the request, which could affect the <formsData> element in the cloned session.)



For more information on asynchronous requests, see *Asynchronous Request Processing* in the *Guide to EXAMPLE Server®*.

PublicRecalc Setting

The *PublicRecalc* configuration setting in the Server.config file can be used to enhance performance of the EXAMPLE Forms™ engine. This setting determines whether the system recalculates public fields on forms or if it simply accesses their values from the session XML.

To set *PublicRecalc* configuration setting:

1. Open the Edit Configuration Utility.
2. In the **Virtual Directory** list, click the desired EXAMPLE Server® directory.
3. Under **Mode**, click **Form**.
4. On the **Server Config** tab, in the list of settings, expand **Forms**.
5. Click **PublicRecalc**.
6. In the **value** text box, enter the desired option:
 - **0** — Do not recalculate public fields on a form during a <ManuScript.getFormsRq> request. Instead, access their value from the Session XML.
 - **1** — Calculate all public fields on a form during a <ManuScript.getFormsRq> request.



For Duck Creek Templates™ forms to function properly, the *PublicRecalc* setting must be set to 1.

7. Click **Save**.
8. Click **Close**.

How Forms Processing Works

The EXAMPLE Forms™ system uses a series of EXAMPLE Server® requests to generate and process forms.

EXAMPLE Forms™ Requests

The EXAMPLE Forms™ system uses two types of EXAMPLE Server® requests to process forms: ManuScript requests, and FormsEngine requests.

ManuScript Requests

The EXAMPLE Forms™ system relies on two ManuScript requests.

Request	Description
---------	-------------

<ManuScript.getFormsRq>	Builds the form and merge field list for a particular print job or form. The response will contain the list of forms to be printed and can have print-ready data for each merge field on the form. For more information, see the <i>EXAMPLE Server® Object Reference</i> .
<ManuScript.printFormsRq>	Generates Portable Document Format (PDF) output files according to the form and merge field list information. The form and merge field list information can be specified in the request by the user or be determined within the call to EXAMPLE Server®. For more information, see the <i>EXAMPLE Server® Object Reference</i> .

The <ManuScript.getFormsRq> request is the only request used for Forms Levels 1 and 2. By contrast, Level 3 Forms uses both ManuScript requests and all FormsEngine requests.

FormsEngine Requests

The Forms Engine component of the EXAMPLE Platform® supports the ability to view and manage forms in a print job. The EXAMPLE Express® system uses the Forms Engine to handle the content of the Policy Forms page and to allow users to view and manipulate the forms to be processed for a policy. The Forms Engine component specifically supports Level 3 Forms production.

The EXAMPLE Forms™ system supports Level 3 Forms production with the following EXAMPLE Server® requests.

Request	Description
<FormsEngine.getChangedFormsRq>	Retrieves a list of forms changes between two policy versions. For more information, see the <i>EXAMPLE Server® Object Reference</i> .
<FormsEngine.initPrintJobRq>	Populates the <formsData> element with the default values of forms and their selection status based on ManuScript rules. If a list already exists in the <formsData> element and the <i>forceInit</i> attribute is set, the existing list will be replaced with the named print job. If the <i>preserveHistory</i> and <i>endorsement</i> attributes are set, only new or modified forms will be included in the list. For more information, see the <i>EXAMPLE Server® Object Reference</i> .
<FormsEngine.listPrintJobRq>	Retrieves the list of forms specified in the <formsData> element, and displays them in EXAMPLE Express®. For more information, see the <i>EXAMPLE Server® Object Reference</i> .
<FormsEngine.processPrintJobRq>	Initiates the <ManuScript.printFormsRq> request, passing attributes through to this request. This request uses the <formsData> element to define the forms printed in the print job. It processes only the forms selected in the Policy Forms page. For more information about the <FormsEngine.processPrintJobRq> request, see the <i>EXAMPLE Server® Object Reference</i> .
<FormsEngine.setPrintJobItemsRq>	Modifies the selection status of items in the active print job list (<formsData>) and indicates forms to be removed. This request modifies only the forms specified in the <printDocs> list. For more information, see the <i>EXAMPLE Server® Object Reference</i> .

<FormsEngine.validatePrintJobRq>

Verifies that all required fields on selected forms contain valid values and returns "valid" or "invalid" accordingly. To be valid a form must have valid data in all required merge fields as defined in Forms Mapper in EXAMPLE Author®. If "invalid," the Forms Engine generates a list of forms with missing fields. For more information, see the *EXAMPLE Server® Object Reference*.

The FormsEngine requests also use the following two requests to set and retrieve information from the <formsData> element in the session:

- <Session.setDocumentRq>
- <Session.getDocumentRq>

For more information about these requests, see the *EXAMPLE Server® Object Reference*.

Form Generation and Storage

The EXAMPLE Forms™ system follows the process below when generating and storing a policy's forms information at runtime:

1. The system executes the <FormsEngine.initPrintJobRq> request, which retrieves the list of forms (*i.e.*, the print job) available for a particular policy. This list of forms is stored in a document in the session file called <_processingData>. Within this document, the forms list resides under the <formsData> element (*_processingData/formsData*).
2. The list of forms is displayed to the user on the Policy Forms page in EXAMPLE Express® using the <FormsEngine.listPrintJobRq> request. From this page, the user can browse, edit, select, add, remove, and preview the desired forms. As the user adds, removes, and marks various forms for printing, the <FormsEngine.setPrintJobItemsRq> request modifies the status of the forms in the <formsData> element.
3. When the user indicates that all form selections have been made, the system saves the information in the <formsData> element to the database.



A user may access the <formsData> element from outside EXAMPLE Server® through the <Session.getDocumentRq> and <Session.setDocumentRq> requests. For more information on these requests, see the *EXAMPLE Server® Object Reference*.

Out-of-Sequence Processing and Forms

When the EXAMPLE Forms™ system is used with EXAMPLE TransACT®, forms may need to be reconciled during out-of-sequence processing. The following forms processes occur during the out-of-sequence reconcile process.

When a form is added during an out-of-sequence transaction:

- The form is added to the FormsXML column for the out-of-sequence transaction, and policy forms are generated for the transaction.
- The form is added to the FormsXML column for each subsequent transaction until the system reaches either of the following:
 - The last transaction on the policy
 - A transaction that already contains the added form

Policy forms for subsequent transactions are NOT regenerated.

When a form is modified during an out-of-sequence transaction:

- The form data is modified in the FormsXML column for the out-of-sequence transaction, and policy forms are generated for the transaction.
- Form data in subsequent transactions is NOT modified, and policy forms for those transactions are NOT regenerated.

Consider the following example:

A policy has the following transaction history.

Transaction	Effective	Forms Included At Committal
New Business	1/1/09	• Form 1 • Form 2 • Form 3
Endorsement	6/1/09	• Form 1 • Form 2 • Form 3
Endorsement	7/1/09	• Form 1 • Form 2 • Form 3 • Form 4

If an out-of-sequence transaction effective on 3/1/09 adds Form 4 and modifies Form 1, the following forms reconciliation will occur:

Transaction		Forms Changes
New Business	1/1/09	• Form 1 • Form 2 • Form 3
Out-of-Sequence Endorsement	3/1/09	• Form 1 is modified in the FormsXML column. • Form 2 remains the same. • Form 3 remains the same. • Form 4 is added to the FormsXML column. • All selected forms are generated.

Endorsement	6/1/09	• Form 1 remains the same. • Form 2 remains the same. • Form 3 remains the same. • Form 4 is added to the FormsXML column.
Endorsement	7/1/09	• Form 1 remains the same. • Form 2 remains the same. • Form 3 remains the same. • Form 4 remains as it was when originally added during this endorsement.

Creating Forms

Creating forms with EXAMPLE Forms™ requires five distinct steps:

1. Creating source documents with merge fields in Microsoft Word or Adobe Acrobat Professional. For instructions on creating source documents, see *Source Documents*.
2. Adding forms and sub-forms (source documents) in Forms Mapper view in EXAMPLE Author®. For instructions on adding forms and sub-forms, see *Forms Mapper in EXAMPLE Author®*.
3. Defining rules for forms and sub-forms in Forms Mapper view in EXAMPLE Author®. For instructions on defining rules for forms and sub-forms, see *Forms Mapper in EXAMPLE Author®*.
4. Mapping Manuscript fields to merge fields on sub-forms in Forms Mapper view in EXAMPLE Author®. For instructions on mapping Manuscript fields to merge fields on sub-forms, see *Forms Mapper in EXAMPLE Author®*.
5. Creating and defining print jobs (groups of forms) in the Forms Mapper view of EXAMPLE Author®. For instructions on creating and defining print jobs, see *Forms Mapper in EXAMPLE Author®*.

Source Documents

When a user's platform license supports complete form-rendering services (Forms Level 3), the EXAMPLE Forms™ system uses source documents.

Source documents are Microsoft Word (.doc) or Portable Document Format (.pdf) documents that contain merge fields. They can be created or modified using a compatible editor that outputs a .doc or .pdf file, respectively. Source documents are added to forms in Forms Mapper as sub-forms. Once they are added in Forms Mapper, EXAMPLE Author® users can map merge fields in the sub-form to specific fields in the Manuscript.

Setting Up the Forms Directory

Source documents must reside within a single directory on the EXAMPLE Platform®. This directory can include a folder structure of any design, and documents may exist in any folder in the directory. When you insert a source document in a Manuscript, EXAMPLE Author® looks for the document under the configured forms directory and stores a relative path to the document within the Manuscript. This relative path allows forms to be deployed from EXAMPLE Author® to EXAMPLE Express® because both components point to the same forms directory. It also allows configuration of the

forms directory location to vary from installation to installation as long as the folder structure within the forms directory remains the same.



Ensure all source documents reside within a single directory on the EXAMPLE Platform®. Duck Creek Technologies, Inc. recommends using the default forms directory configured during installation. If you must change the location of the forms directory, however, you must set it in three locations. For additional information, see *Configuring the EXAMPLE Forms™ Root Path in EXAMPLE Author®* and *Configuring the EXAMPLE Forms™ Root Path in Server.config in EXAMPLE Forms™ Installation*.

Creating Source Documents

Source documents can be created as sub-forms in Forms Mapper, or they can be added as merge fields before the source document is created. Either way, the source document must be created in Microsoft Word or Adobe Acrobat Professional.

In Microsoft Word, the merge fields or Mail Merge tools are used to create the source document. If the document is configured to utilize a data source, the mail merge tool should be used. If a data source has not been configured, the merge field tool should be used. For more information about using merge fields, see *Creating a Merge Field Document in Microsoft Word*. For more information about using the Mail Merge tool, see *Creating a Mail Merge Document in Microsoft Word*.

To create a source document after defining a form in Forms Mapper, create the merge fields using a .csv file and the Mail Merge function in Microsoft Word. In Adobe Acrobat Professional, use the Form Field tool. For more information about using the Form Field tool, see *Creating a Form Field Document in Adobe Acrobat Professional*.

Choosing the Type of Source Document

You can determine the type of source document to use based on your needs for specific forms. When your desired output requires a fixed format and you wish to generate .pdf output files, use .pdf documents. When your desired output requires variable content or formatting, or if you wish to generate .doc, .rtf, or .htm output files, use .doc documents. (You can also generate .pdf output from .doc source documents.) With .doc sub-forms, you can define multiple source document components in such a way that the parts are assembled into the desired output at runtime according to established rules and iterations. While you can add multiple .doc sub-forms to a form in Forms Mapper, you can only add one .pdf sub-form to a form.



Use Microsoft Word (.doc sub-forms) when creating multiple sub-forms to add to a form in Forms Mapper. While you can add multiple .doc sub-forms to a form in Forms Mapper, you can add only one .pdf sub-form to a form.



Use a table or a .pdf source form if more than one column is needed in a source form. EXAMPLE Forms™ only supports one column in .doc source forms.

Consider the following limitations of each type of source document. In addition, see *Building Variable Output Forms* and *Building Iterating Forms* for recommendations on handling special types of forms.

.Doc Source Documents

EXAMPLE Forms™ supports all basic functions of word processing in source documents, including boldface, italics, underlining, colors, paragraph indentations, margins, text alignment, images, headers, footers, and tables.

EXAMPLE Forms™ does not, however, support the following advanced word processing features of Microsoft Word.

Item	Description
Form Fields	Form fields are items such as check boxes and text fields that can normally be placed in a .doc document.  Use square brackets, such as [], for a check box instead of form fields in source documents. You can instruct the system to place an X in the brackets if the item should be checked. Do not use text fields in a source document.
Multiple Columns	EXAMPLE Forms™ only supports one column in .doc source forms. Instead of multiple columns, use a single column, a table, or a .pdf source form.
Multiple Headers and Footers	You cannot use multiple headers and footers in a .doc source document.
Merge Field Length	Unlike Microsoft Word XP, EXAMPLE Forms™ does not limit the length of merge field names to 38 characters. When using Word XP, however, you must limit the merge fields in a Manuscript to 38 characters in order to match the Word output. Other versions of Microsoft Word allow you to use more characters.
Pagination	EXAMPLE Forms™ does not support the format "Page X of Y" (where "X" equals the page number and "Y" equals the total number of pages in the form) when paginating forms. Instead, use the format "Page X" or simply display the number.

.Pdf Source Documents

.Pdf files work best when the structure of the output form needs to be static. Building or modifying .pdf source documents requires a .pdf editor.

.Pdf source documents have the following limitations:

- Forms Mapper will not display .pdf forms. To view a .pdf sub-form, right-click the desired sub-form, and then click **View PDF** to open a version of the form in Adobe Acrobat.



Do not use Windows Journal Viewer and Adobe Acrobat 6.0 on Windows XP when attempting to view a PDF document in Forms Mapper. A conflict between the Windows Journal Viewer and Adobe Acrobat 6.0 on Windows XP could cause the View PDF feature in Forms Mapper to fail.

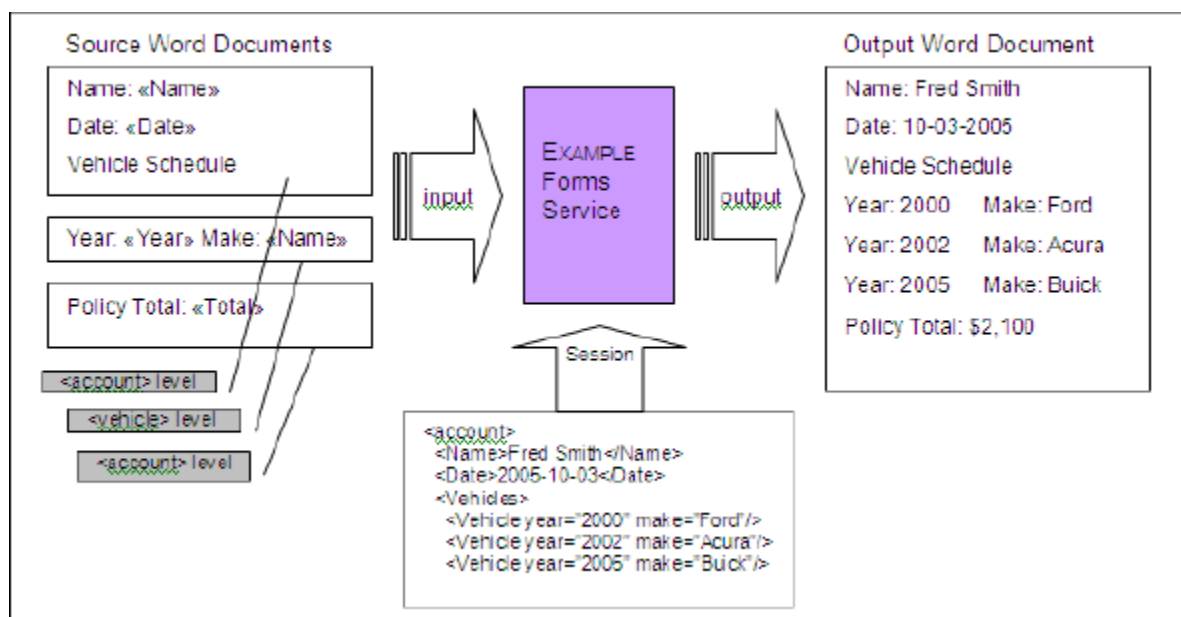
- A .pdf form cannot contain more than one .pdf sub-form. As a result, you cannot use .pdf source documents to build dynamic iterating tables.

- Merge fields on a .pdf sub-form cannot be edited within Forms Mapper. You can, however, use Forms Mapper to open the sub-form in a .pdf editor (Adobe Acrobat Professional) where you can then edit the merge fields.
- If you set a specific font size for data within the merge fields on a .pdf form, data will be truncated to fit within the merge fields. Setting the font size to *Auto* will cause the data to shrink or expand to fill the merge field box.
- EXAMPLE Forms™ does not support the format “Page X of Y” when creating sub-forms.

Building Variable Output Forms

Some forms may need to be assembled from separate conditional parts, or a section of a form may need to repeat based on the number of iterating items in that section. To accommodate this need, you can create a single .doc file for each section of a form and then add each .doc file as a sub-form in Forms Mapper. With this technique, you can define rules and iteration support for each section of the form, building the output dynamically. (The EXAMPLE Forms™ system supports variable output for .doc forms only.)

The figure below demonstrates a variable output form containing two non-iterating and one iterating sub-form. The first sub-form, which relates to the Account group in the Manuscript, does not iterate. The second sub-form, which relates to the Vehicle group in the Manuscript, iterates. This sub-form repeats for each vehicle in the policy. The final sub-form, which again relates to the Account group, does not iterate.



A few limitations exist for multiple sub-forms within a single form:

- A form can contain only one .pdf sub-form.



Keep sub-form document formats consistent within forms. A form cannot contain both .doc and .pdf sub-forms.

- Only one header and one footer can be used within a form. While multiple headers and conditions for those headers can be configured for a print job, only one header will be used. This header will appear on all sub-forms in that form.
- Only one paper size, margin, and orientation can be specified for a form. The paper size, margins, and orientation from the first sub-form will be used for all sub-forms in that form.

Building Iterating Forms

Many times, iterations of particular fields need to be displayed on a form. For example, if an auto policy has multiple autos with a premium for each, each auto and premium will need to appear on a form. You can accommodate situations like this with fixed or dynamic tables.

Fixed Tables

A fixed table has a predetermined number of rows and columns. For an auto policy that only allows three autos on a page, for example, three lines appear on the printed form to display the information. If a policy contains fewer than three autos, any unused lines are left blank. This allows the form to have a fixed format and prevents page overflow issues. In a fixed table, all possible fields must be mapped as many times as they are allowed to appear on a page. If additional fields exist in a policy, a second page will be generated. Fixed tables work best with .pdf source forms.

Creating a Fixed Table Document

To create a fixed table:

1. In the source document for a form, create a table with the desired number of rows and columns. The number of rows determines the number of iterations that can appear on the form.
2. Insert merge fields for each possible iteration, giving each merge field a distinct name. For example, use **VehicleModel1**, **VehicleModel2**, **VehicleModel3**, and so on.



For information on inserting merge fields into source documents, see *Creating a Merge Field Document in Microsoft Word* or *Creating a Form Field Document in Adobe Acrobat Professional*.

Mapping Fields to a Fixed Table

When using a fixed table, if you need a set number of iterations to appear on a page, you must create and map a uniquely named merge field for each iteration.

To map fields to a fixed table:

1. In Forms Mapper in EXAMPLE Author®, open the sub-form with the fixed table.



If you have not added the source document with the fixed table as a sub-form in Forms Mapper, you must do so. For information on adding a sub-form, see *Adding a Sub-form*.

2. In the **Group** grid, set the **# of Iterations** for the group that contains the iterating fields to the fixed number of iterations present in the table.



For instructions on setting options for groups, see *Setting Group Properties*.

3. Map a field in the Manuscript to each merge field in the sub-form. Map the same field in the Manuscript to each iteration of that field on the sub-form.
EXAMPLE Author® and EXAMPLE Express® will handle this repeated mapping by automatically specifying an Index for each merge field. This Index is indicated in the column to the right of the *Mapped Field* column in the Merge Fields grid. The Index number and merge field position (first iteration, second iteration, third iteration, and so on) should correspond. If necessary, you can edit the Index number.

Adding Iterations to a Fixed Table

A fixed form can display more than its set number of iterations even though the page is restricted to display only that set number. If the policy has more iterations than the fixed table allows, the extra iterations print to another page.



Adjust settings for iterations in a fixed table to ensure information displays properly. Extra iterations added to a fixed table will print to another page. To ensure that information such as policy number, name, and address for the account appear on every page, you may need to adjust settings for the various groups on the form. For additional information, see *Adding Iterations to a Fixed Table* in *EXAMPLE Forms™ Reference Guide*.

To accommodate extra iterations in a fixed table:

1. In Forms Mapper view in EXAMPLE Author®, select the sub-form with the fixed table from the **Forms** list.
2. On the **Group** tab, change the **End Iteration** for the groups containing the fields you want to appear on every page (such as account or premium information) to **1**.
Normally, you want the information displayed on a form only once to allow for multiple iterations. You achieve this by setting the **Start Iteration** to **1** and **End Iteration** to *****. When you need the same data displayed on multiple pages, however, you must change the **End Iteration** to **1**.

The fields will now appear on every page.

Dynamic Tables

When a table may contain an unknown number of iterations, a dynamic table can be used. A dynamic table form allows you to create a policy with an unlimited number of iterations. Dynamic tables will only work with .doc source documents. For more information about source documents and tables, see *Creating Source Documents* in the *EXAMPLE Forms™ Reference Guide*.

A dynamic table has a predetermined number of columns but a variable number of rows. This format allows you to create a policy with an unlimited number of iterations. With a dynamic table, you only need to map the fields for the iterating item once. Dynamic tables will only work with .doc source documents.



You cannot use dynamic tables in .pdf documents.

To create a dynamic table document, you must break the document into two or more parts. Iterative parts of the form must be separated from non-iterative parts of the form.

To create a dynamic table document:

1. Separate iterative and non-iterative parts of the desired document and save each part as a separate .doc file.
2. In Forms Mapper in EXAMPLE Author®, create a form to hold the source documents you just created.

For information on creating forms, see *Adding a Form*.

3. Create sub-forms within the form you just created using the separate .doc files created in Step 1. Be sure these sub-forms are in the proper order. If necessary, use the **Move Up** button  or **Move Down** button  to change the order.

For information on creating sub-forms, see *Adding a Sub-form*.

4. Select the first sub-form in the list, and then map the appropriate Manuscript fields to the fields in the source document. You only need to map iterating fields once.
5. Repeat step 4 for every sub-form in the list.
6. To any sub-form with iterating fields, add the group through which the fields in that sub-form iterate. In some cases, you may also need to add groups to the form containing the sub-form.

For information on adding a group, see *Adding a Group*.

Tips for Fixed and Dynamic Tables

The following tips may help when accommodating iterating forms.

- When creating a .pdf source form with a fixed table, create the table in Microsoft Word, and then convert the form to Adobe Acrobat. Add the merge fields to the .pdf document. It is much easier to build tables in Microsoft Word than in Adobe Acrobat.
- Plan source documents to accommodate varying merge field and table cell lengths when data is merged onto the form. Remember that the length of merge fields and table cells and the width of rows and columns may vary when data is merged onto the form.
- When creating a .pdf source form with a fixed table, create the table in Microsoft Word, and then convert the form to Adobe Acrobat. Add the merge fields to the .pdf document. It is much easier to build tables in Microsoft Word than in Adobe Acrobat.
- When adding iterating groups to a dynamic table, add the group through which the fields in that sub-form iterate to the sub-form. Add all other groups to the form.
- When building a dynamic table, you can build a table with outlined rows by including a border around your single iterating row in the source document.

Using Image Merge Fields

In addition to text merge fields, you can also add image merge fields to a source document. An image merge field allows you to map an image to a form. In EXAMPLE Author®, an image merge field is mapped to a field that contains the path to the desired image.

To add an image merge field add an *img:* prefix to the merge field name in the source document.



When using image merge fields, the size of the printed image will be the physical size of the image specified in the Manuscript field.

Using Electronic Signatures

A third-party product is required to use electronic signatures.

Creating a Merge Field Document in Microsoft Word

In .doc source documents, merge fields can be created using the merge field or mail merge tools in Microsoft Word. If the document is configured to utilize a data source, the mail merge tool should be used. If a data source has not been configured, the merge field tool should be used.

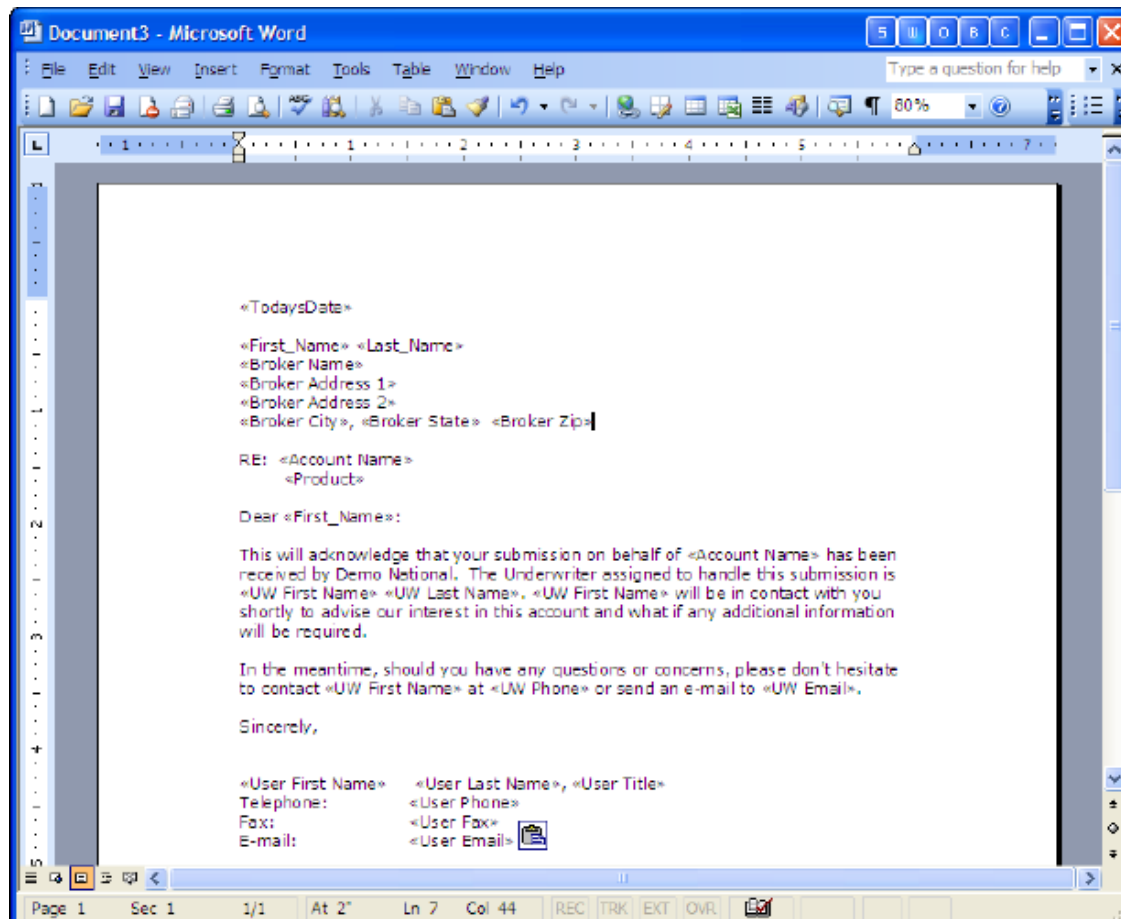


Although .doc source documents can be created using Microsoft Word, EXAMPLE Forms™ uses TXTextControlWords.exe to generate .doc forms. Preview all .doc forms using TXTextControlWords.exe to ensure proper generation of forms. For more information about previewing .doc source documents using TXTextControlWords.exe, see *Previewing in TXTextControlWords.exe*.

To create a merge field document:

1. Open an existing Microsoft Word document or create a new document.
2. Position the cursor where the merge field should be inserted in the document.
3. Perform one of the following:
 - In Microsoft Word 2003:
 - On the **Insert** menu, click **Field**.
 - In Microsoft Word 2007 or 2010:
 - On the **Insert** tab click **Quick Parts** and then click **Fields**.
4. In the **Field names** list, click **MergeField**.
5. Enter the appropriate name in the **Field name** text box below the **Field properties** heading.
6. Click **OK**.
7. Repeat steps 2 through 6 for each required field.
8. Save the document in the forms directory defined by the *Path* configuration setting in the Server.config file and in the EXAMPLE Author® Local and Enterprise Options.
9. Preview the document with TXTextControlWords.exe. For information on using this tool, see *Previewing in TXTextControlWords.exe*.

Merge fields in a Word document should appear similar to the following illustration.



10. Add the document as a sub-form in Forms Mapper.

For more information about adding a sub-form, see *Sub-forms*.

Formatting Tips for Microsoft Word Source Documents

The following are tips for formatting source documents in Microsoft Word.

- **Page Breaks** — Press **CTRL+ENTER** to insert a hard page break and start a new page.
- **Aligning Text** — For accurate text alignment, use one of the following options:
 - **Tab Stops** — Use to left-justify columns of text.
 - **Tables** — Use to right-justify some columns and left-justify other columns.



Do not use the spacebar to align text in a source document. Although it may display correctly in the source document, using the spacebar to align text may not be exact and can result in formatting errors when generating forms.

- **Justifying Text** — Use the appropriate toolbar setting to right-justify or center text.

Creating a Form Field Document in Adobe Acrobat Professional

You can build .pdf source documents with the Form Field tool in Adobe Acrobat Professional. To create .pdf files containing merge fields, you must have Adobe Acrobat Professional or a third-party application with similar features. If you do not have Adobe Acrobat Professional, consult the help information specific to your application for instructions on creating merge fields in .pdf documents. Not all third-party .pdf applications will work with EXAMPLE Forms™.

To create a Form Field document in Adobe Acrobat:

1. In Adobe Acrobat Professional, on the **File** menu, click **Open**.
The Open dialog box appears.
2. Browse to the desired .pdf document, and then click **Open**.
3. On the **Tools** menu, point to **Advanced Editing**, and then click **Text Field Tool**.
4. Place the crosshair cursor where you want a field to appear on the form, and then double-click to create the field.
The Text Field Properties dialog box appears.
5. Click the **General** tab.
6. Enter a **Name** for the field, and then click the **Appearance** tab.



You can add an image to a merge field using an *img:* prefix. When an *img:* prefix is attached to a merge field's name, EXAMPLE Forms™ recognizes the corresponding mapped field's value as a path to the appropriate image.

7. Enter the **Font Size**.



Do not set the font size to Auto when creating form field documents in Adobe Acrobat. The Auto setting causes font size to vary according to the number of characters entered. In most cases, set the font size to 10.

8. Click the **Options** tab.
9. Clear all check boxes.



Selecting **Multi-line** will wrap text until it fills the set size of the form field. After reaching the maximum, no additional characters can be entered. Selecting **Scroll long text** will allow the field to expand to accommodate data, but it will truncate the text when viewed and printed. You can click a plus sign next to the field to scroll through additional information in the field, but the additional text does not print. EXAMPLE Author® does not support scroll bars or plus signs in .pdf files.

10. (Optional) To adjust text alignment within the field, in the **Alignment** drop-down list, click the appropriate setting.
11. Click **Close**.
12. Repeat Steps 3-11 for each form field you wish to add.

- When finished, save the document in the forms directory defined by the *Path* configuration setting in the Server.config file and in the EXAMPLE Author® Local and Enterprise Options. Then add the document as a sub-form in Forms Mapper.

Adobe documents with Form Fields should appear similar to the illustration below.

Adobe Acrobat Professional - [CG04350305.pdf]

File Edit View Document Comments Tools Advanced Window Help

Search Create PDF Comment & Markup Send for Review Secure Sign Ports

This document contains interactive form fields. Highlight fields

POLICY NUMBER: [PolicyNumber] COMMERCIAL GENERAL LIABILITY CG 04 35 03 06

THIS ENDORSEMENT CHANGES THE POLICY. PLEASE READ IT CAREFULLY.

EMPLOYEE BENEFITS LIABILITY COVERAGE

THIS ENDORSEMENT PROVIDES CLAIMS-MADE COVERAGE. PLEASE READ THE ENTIRE ENDORSEMENT CAREFULLY.

This endorsement modifies insurance provided under the following:

COMMERCIAL GENERAL LIABILITY COVERAGE PART

SCHEDULE

Coverage	Limit Of Insurance	Deductible	Premium
Employee Benefits Programs	\$[EmployeeBenefitsPer] each employee aggregate	\$[EmployeeBenefitsDed]	\$[EmployeeBenefitsPremium]
Retroactive Date:	[EmployeeBenefitsRetroDate]		

Information required to complete this Schedule, if not shown above, will be shown in the Declarations.

A. The following is added to Section I – Coverages:
COVERAGE – EMPLOYEE BENEFITS

No other obligation or liability to pay sums or perform acts or services is covered unless explicitly provided for under Coverage

Using a .Csv File

You can use a .csv (Comma Separated Values) file to help set up merge fields in a Microsoft Word source document after defining a form and its merge fields in Forms Mapper. The .csv file stores values found in the Merge Field column in Forms Mapper.

A .csv file can be helpful in two situations:

- When creating a source document after mapping merge fields and defining rules for a sub-form in Forms Mapper.
- When you have added merge fields to a sub-form in Forms Mapper that have not yet been added to an existing Microsoft Word source document used for that sub-form.



While you can manually create merge fields in a source document that match newly created merge fields in Forms Mapper, you may find it easier to use a .csv file. A .csv file will ensure the merge field names in the Microsoft Word document match the merge field names in Forms Mapper. You cannot use a .csv file with .pdf source documents.

To use a .csv file, you must export a form created in Forms Mapper to a .csv file using the **Export to CSV** function. You must then use the .csv file in conjunction with the Mail Merge tool in Microsoft Word. Exporting to a .csv file can only be done if you have purchased an EXAMPLE Forms™ license (Forms Level 3).

Exporting to a .Csv File

After defining the rules and merge fields for a specific form in Forms Mapper, you may export the form to a .csv file.

To export to a .csv file in Forms Mapper:

1. On the **Edit** menu, click **Export to CSV**.
The Save As dialog box appears with the name of the form pre-filled in the **File name** text box. If it is not pre-filled, name the .csv file.
2. Click **Save**.



Be sure to save the file in the Forms Directory defined in the Edit Configuration Utility and the EXAMPLE Author® Local and Enterprise Options.

When you use Microsoft Word to modify a source document, this .csv file will be the data source for the mail merge document.

Creating a Mail Merge Document in Microsoft Word

In .doc source documents, merge fields can be created using the merge field or mail merge tools in Microsoft Word. If the document is configured to utilize a data source, the mail merge tool should be used. If a data source has not been configured, the merge field tool should be used. You can create a mail merge document in Microsoft Word and add it to the Manuscript as a sub-form.



Although .doc source documents can be created using Microsoft Word, EXAMPLE Forms™ uses TXTextControlWords.exe to generate .doc forms. Preview all .doc forms using TXTextControlWords.exe to ensure proper generation of forms. For more information about previewing .doc source documents using TXTextControlWords.exe, see *Previewing in TXTextControlWords.exe*.

To create a mail merge document:

1. Open an existing Microsoft Word document or create a new one.
2. Perform one of the following:
 - In Microsoft Word 2003:
 - On the **Tools** menu, click **Letters and Mailings**, and then click **Mail Merge**.
 - In Microsoft Word 2007 or 2010:

- On the **Mailings** tab, click **Start Mail Merge**, and then click **Step by Step Mail Merge Wizard**.

The Mail Merge task pane appears and guides you through setting up the document. In step 3, you are prompted to **Select Recipients**.

3. Click **Use an existing list**, and then click **Next**.
4. Browse to and open the appropriate existing .csv file.
5. Click a **Field** delimiter and a **Record** delimiter.
6. Click **OK**.
7. Click the first line in the **List of recipients**, and then click **OK**.
8. Perform one of the following:
 - In Microsoft Word 2003:
 - On the **View** menu, click **Toolbars**, and then click **Mail Merge**.
 - Position the cursor where the merge field should be inserted in the document, and then click the **Insert Merge Fields** button  on the **Mail Merge** tool bar.
 - Click the appropriate merge field and then click **Insert**.
 - Click **Close**.
 - In Microsoft Word 2007 or 2010:
 - Position the cursor where the merge field should be inserted in the document, and then click **Insert Merge Field** on the **Mailings** tab.
 - Click the the appropriate field.
9. Repeat the appropriate steps until all required merge fields are inserted in the document.
10. Save the document in the Forms Directory defined in the Edit Configuration Utility and the Local and Enterprise Options in EXAMPLE Author®.
11. Preview the document with TXTextControlWords.exe. For information on previewing a document using TXTextControlWords.exe, see *Previewing in TXTextControlWords.exe*.
12. Add the document as a sub-form in Forms Mapper.

For more information about adding a sub-form, see *Sub-forms*.

Previewing in TXTextControlWords.exe

The forms generated by the EXAMPLE Forms™ engine are created using TXTextControlWords.exe rather than Microsoft Word. To ensure .doc source document forms generate as expected, preview .doc source documents using TXTextControlWords.exe.

To preview a source document in TXTextControlWords.exe:

1. In the Duck Creek root directory (for example, C:\Program Files\Duck Creek Technologies), navigate to the Shared/Bin directory.
2. Locate and double-click **TXTextControlWords.exe** to open the program.
3. Browse to the appropriate file location, and open the .doc source document to preview.
4. Verify that the document is formatted as expected.
5. (Optional) Make any necessary changes, and then click **Save** on the **File** menu.

6. Click **Exit** on the **File** menu, or close the window.

Forms Mapper in EXAMPLE Author®

The Forms Mapper view is one of five views available in EXAMPLE Author®. In Forms Mapper, you can define rules for forms, map Manuscript fields to forms, and create and define print jobs. To enable this process, you define the fields in the Manuscript that will generate the form information, and Forms Mapper stores the data in an XML document. Duck Creek Templates™ Manuscripts should already have the fields needed to process each form listed in Forms Mapper.

With Forms Mapper, you can define:

- Properties that describe how and when a form or sub-form is generated.
- Merge fields that associate Manuscript fields with specific locations on a form.
- Formats for information that appears on a form.
- Settings for the groups through which the program iterates for a form or sub-form.
- Watermarks and stamps to add to a form (when generating .pdf output files).
- Print jobs (specific sets of forms).

You define these items through five different components of Forms Mapper:

- Forms
- Sub-forms
- Merge fields
- Groups
- Print jobs

For instructions on creating and defining forms and print jobs in Forms Mapper, see the *Guide to EXAMPLE Author®*.

Enabling Form Generation and Printing

In EXAMPLE Express®, users can view and generate printable forms using various actions that initiate forms requests to the EXAMPLE Server®. These actions can be custom actions available during the interview process or actions found on the default EXAMPLE Express® pages.



When a user generates forms, EXAMPLE Express® does not actually print the forms on a printer. Instead, EXAMPLE Express® and the EXAMPLE Forms™ engine support the ability to attach forms to policy, save them to a folder, or e-mail them. Forms can then be sent to a configured printer, or the Duck Creek Technologies, Inc. EXAMPLE Form Print utility can automatically print forms sent to a specified folder.

To enable the generation and printing of forms for a product, you must:

- Configure the product to launch the Policy Forms page. For information, see *Launching the Policy Forms Page*.
- Configure the product to prevent issuance until forms have been selected. For information, see *Enforcing Form Selection*.
- Configure the `<FormsEngine.processPrintJobRq>` request. For information, see *Configuring the <FormsEngine.processPrintJobRq>*.
- (Optional) Modify the control file to include any desired fields. For information, see *Modifying the Control File*.

- (Optional) Configure the EXAMPLE Form Print utility to print forms sent to a specified folder. For information, see *EXAMPLE Form Print*.

Launching the Policy Forms Page

To select which forms to generate for a given policy, an EXAMPLE Express® user must access the Policy Forms page. This page can be accessed through a custom action in the policy interview process. Custom actions are created in Page Designer view in EXAMPLE Author®.



Duck Creek Technologies, Inc. automatically configures a custom action for Duck Creek Templates™ products (in the Product Base Data Manuscript). Customers that do not inherit from the Product Base Data Manuscript can either configure this action themselves or commission Duck Creek Technologies, Inc. do so for them.

To create a custom action that launches the Policy Forms page:

- In the Page Designer view of a Manuscript with a print job, add an action to the desired page using the following values.

Property	Value
Action Type	Custom Action
Caption	"Policy Forms"
Express Action	"previewAsyncPrintJob"
Print Job	Field containing the print job name.
Print Job Manuscript	Field containing the Manuscript ID where the print job is located.



For more information on creating an action, see *Actions* in the *Guide to EXAMPLE Author®*.

Enforcing Form Selection

With EXAMPLE Forms™, a user cannot issue a policy and move into EXAMPLE TransACT® until forms have been selected on the Policy Forms page. To ensure the user selects the forms to attach to a policy before completing the policy, a condition must be attached to the **Complete Issuance** button (or any button used to bind the policy) that prevents the user from binding the policy before he or she has visited the Policy Forms page and selected the appropriate forms. This restriction can be achieved through a Show condition on the button. If the user has not visited the Policy Forms page and selected forms for the policy, the **Complete Issuance** button will not appear.

Setting up the Show condition for the **Complete Issuance** button requires the creation of a few Manuscript fields. If your Manuscript inherits from the Duck Creek Technologies, Inc. Product Base Data Manuscript, the fields required to implement the Show condition are already set up in your Manuscript. If your Manuscript does not inherit from the Product Base Data Manuscript, however, you must configure the following fields yourself or commission Duck Creek Technologies, Inc. to do so for you.

Retrieving the Print Job

To configure the show condition, your Manuscript must contain three fields: one that iterates through the **FormsData** group in the Manuscript to retrieve the appropriate print jobs, and two that ensure the print job retrieved is the one desired.



If your Manuscript inherits from the Duck Creek Technologies, Inc. Product Base Data Manuscript, these fields will already be set up in your Manuscript. If your Manuscript does not inherit from the Duck Creek Technologies, Inc. Product Base Data Manuscript, however, you must configure the following fields yourself or commission Duck Creek Technologies, Inc. to do so for you.

Retrieving the Print Job Name

To retrieve the print job name:

1. Create a field in the **SystemForms** group using the following values.

Property	Value
Field Name	ActivePrintJobName (or any desired name)
Visibility	Private
Data Type	String
Value	Iterator

2. Specify the following iterator values for the field.

Property	Value
Iterate through	the first iteration
of the group	FormsData
to find the	sum
of all	FormsData.printJobName

Checking for the Correct Print Job

To check for the correct print job:

1. Create a table lookup field called **PrintingAndFormsData.PrintJobName** (or any desired name) that determines the correct print job to use.
2. Create a Boolean field in the **SystemForms** group using the following values.

Property	Value
----------	-------

Field Name	IsIssuePrintJobActive (or any desired name)
Visibility	Private
Data Type	Boolean
Value	Conditional
Condition	SystemForms.ActivePrintJobName = PrintingAndFormsData.PrintJobName

This field is initiated by the **PolicyOutputNonShredded.FormsCompleted** field. This field is one of the Show conditions on the **Complete Issuance** button that appears on the Submission page in EXAMPLE Express®. Because the Show condition runs every time the Submission page refreshes, **SystemForms.IsIssuePrintJobActive** and the fields initiated by its rules run every time the Submission page refreshes.

Creating the Show Condition

The **PolicyOutputNonShredded.FormsCompleted** field is one of the Show conditions on the **Complete Issuance** button that appears on the Submission page in EXAMPLE Express®. It calls the **SystemForms.IsIssuePrintJobActive** field.



If your Manuscript is an Duck Creek Templates™ Manuscript, or if it inherits from the Duck Creek Technologies, Inc. Product Base Data Manuscript, this field will already be set up in your Manuscript. If your Manuscript does not inherit from the Duck Creek Technologies, Inc. Product Base Data Manuscript, however, you must configure the following field yourself or commission Duck Creek Technologies, Inc. to do so for you.

To create the **PolicyOutputNonShredded.FormsCompleted** field:

1. Create a field in the **PolicyOutputNonShredded** group with the following values.

Property	Value
Field Name	FormsCompleted
Visibility	Public
Data Type	Boolean
Value	N/A
Default	Constant/Field: False
Misc. Rules	(defined)

2. Define the following miscellaneous rule for the field.

Condition	Then
-----------	------

SystemForms.IsIssuePrintJobActive = True
--

True

If this field and all other fields attached to the Show condition return a value of *True*, the **Complete Issuance** button will appear on the page. A user can click this button to complete a policy, and the appropriate policy forms will be processed.

Configuring the

After initializing an active print job with the `<FormsEngine.initPrintJobRq>` request and selecting the desired forms on the EXAMPLE Express® Policy Forms page, the EXAMPLE Express® user must initiate "printing"—or generation—of the forms. To set up printing in EXAMPLE Express®, you can create a custom action. Duck Creek Technologies, Inc. automatically configures this action in the Product Base Data Manuscript. Customers that do not inherit from the Product Base Data Manuscript can either configure the button themselves or commission Duck Creek Technologies, Inc. to do so.



In the Product Base Data Manuscript, users can only access this custom action on the Submission page after they have opened the Policy Forms page, selected or cleared any desired forms, and then clicked **Return to Policy**. Users must have opened and returned from the Policy Forms page *first*. They can then see and click the **Complete Issuance** button to complete the policy and generate the selected forms.

If desired, you can configure your own action to initiate this process using the `<FormsEngine.processPrintJobRq>` request. EXAMPLE Forms™ uses the `<FormsEngine.processPrintJobRq>` request to print the selected forms in a print job. This request invokes the `<Manuscript.printFormsRq>` request, but unlike the `<Manuscript.printFormsRq>`, the `<FormsEngine.processPrintJobRq>` request prints only those forms in the print job the user has marked for printing on the Policy Forms page.

To configure a button to print the active print job through the `<FormsEngine.processPrintJobRq>` request:

1. In the Page Designer view of a Manuscript with a print job, add an action to the desired page using the following values.

Property	Value
Action Type	Goto Page
Caption	"Print" (or whatever you prefer)
Show Condition	PolicyOutputNonShredded.FormsCompleted
	 This property requires a boolean field that tests whether the print job is active. You can find instructions for creating this PolicyOutputNonShredded.FormsCompleted field in <i>Enforcing Form Selection</i> .



For more information on creating an action, see *Actions* in the *Guide to EXAMPLE Author®*.

2. Configure the action to execute the <FormsEngine.processPrintJobRq> request (or the Manuscript field that calls this request), specifying any desired request parameters. When generating forms, you can configure the system to:
 - **Attach forms to a policy** — Attaches forms to the current policy. You can view and print attached forms from the Policy Details page for the given policy.
 - **Save forms to a folder** — Stores forms to a specified folder. If desired, the system can create a new subfolder for every new policy submitted.
 - **Send forms in an e-mail message** — E-mails forms to a specified user.

For more information on the <FormsEngine.processPrintJobRq> request, see the *EXAMPLE Server® Object Reference*.



Currently, the EXAMPLE Forms™ product does not send forms directly to a printer. Duck Creek Technologies, Inc. does, however, offer a print utility that recognizes and prints generated forms on configured printers. For more information, see *EXAMPLE Form Print*.

Modifying the Control File

A control file is an optional file that can accompany a print job. You use a control file to define the order and location of forms that are to be printed. The control file provides information to a print service that aids in printing the forms. You can use the control file to pass information to a third-party integration system. Control files only accompany forms that are saved to a folder or e-mailed.

A control file is stored in the same folder as the forms it contains. The name of the control file is the name of its associated print job with *-order.xml* appended. If you choose to package all the forms in a print job into one .zip file, the control file is included in the zipped file.

You can modify the control file to identify a block of additional control information. For example, you can specify how many copies of a form to print or to which printer to send them. You can also identify a list of fields to write out. This passes other information to the application that is responsible for processing the forms. If you are attached to a database, the values for username, e-mail, and full name are added to the control file from the database.

You can add additional information to the control file by setting the control block in EXAMPLE Author®. In the Control Block dialog box, you can add Manuscript fields that will in turn be added to the control file. Control block data can facilitate integration and identification, because it is used by other systems to identify the policy, policy number, or any other identifier in a given print job. You can only edit the control block data if you have purchased the EXAMPLE Forms™ license.

To add fields to the control block:

1. In Forms Mapper in EXAMPLE Author®, on the **Edit** menu, click **Edit Control File List**. The Control Block dialog box appears.
2. Click **Add**. The Reference Select dialog box appears.

3. Click the field you wish to include in the control block, and then click **OK**.
4. Repeat steps 2 and 3 for each field you wish to include.
5. When finished, click **OK**.

For each field in the list, the following XML will be added to the control file for the print job:

```
<ControlBlock>
  <[fieldname] value=""/>
  <[fieldName] value=""/>
</ControlBlock>
```

To include the control file in a print job, you must specify that you want it created when you send the `<ManuScript.printFormsRq>` (or `<FormsEngine.processPrintJobRq>`) request. To do this, set the `controlFile="1"` request parameter in the ManuScript field calling the request.

EXAMPLE Form Print

With the Duck Creek Technologies, Inc. EXAMPLE Form Print utility, system administrators can automate the printing of forms generated by the EXAMPLE Platform®. This utility uses an installed copy of Adobe Acrobat Reader and is part of the Forms extension of EXAMPLE Server®.

The EXAMPLE Form Print utility automates the printing of forms generated by the EXAMPLE Forms™ system. This utility is part of Level 3 Forms and can be configured to support the form printing needs of any user. EXAMPLE Form Print works by allowing administrators to configure one or more watched folders. The utility then scans these folders at a predetermined frequency. When a .pdf or .doc file is found within a watched folder, the utility sends the file to the designated printer for printing, performs the necessary notification and logging, and then deletes or archives the file.

The EXAMPLE Form Print utility can run on a machine independently of the EXAMPLE Platform®, but the folder in which generated documents are stored must be accessible by both the EXAMPLE Platform® machine and the independent machine. While operating, the EXAMPLE Form Print status window shows which files are being printed and where they are printing.

To begin using EXAMPLE Form Print, you must first perform the following steps:

- Create watched folders.
- Assign printers for duplex and non-duplex printing.
- Assign the path to the Adobe Reader on your system.
- Start the service.



There is a conflict between the Windows Journal Viewer and Adobe Acrobat 6.0 on Windows 2000/XP that could cause EXAMPLE Form Print to fail. If you have the Windows Journal Viewer installed, you will need to correct this issue prior to attempting to use EXAMPLE Form Print. Instructions for correcting this problem can be found on the Adobe web site at <http://www.adobe.com>.

For more information about the Form Print Utility, see *EXAMPLE Form Print*.

Building a Forms Schema File

Schema files are XML documents that express shared vocabularies and allow machines to carry out pre-defined rules. They provide a means for defining the structure, content, and semantics of XML documents. In EXAMPLE Forms™,

the forms schema file defines the print job elements, their order, and the forms contained within the print jobs in a Manuscript. It documents the forms and the fields used by those forms within a specific Manuscript. It also provides the foundation for a relational database of forms, fields, and Manuscripts that can help verify Manuscript maintenance, validate testing, and generate reports.

To build a forms schema file:

1. On the **Edit** menu, click **Generate Forms Schema**.
The Save As dialog box appears.
2. Browse to an appropriate location, and then enter the **File name**.
3. In the **Save as type** list, click the appropriate format and language :
 - **Element/Value or Name/Value** — The Element/Value format places all merge fields in a <dataElement> tag. The Name/Value format places each merge field in an element name matching the merge field. Clicking the Element/Value format sets the *elements* attribute of the <Manuscript.getFormsRq> request to 1. Clicking the Name/Value format sets the *elements* attribute to 0.
 - **XSD or XDR specification language for the schema** — XSD (XML Schema Definition) has become the standard schema language endorsed by the World Wide Web Consortium. XDR (XML Data Reduced) is a deprecated Microsoft-specific schema language. Both formats describe the same set of data. If you do not know which language to use, ask the person or organization who receives the information.
4. Click the appropriate file type, and then click **Save**.

Managing Forms

You can manage the forms created for various policies by organizing them in folders. To do this, you set a Manuscript to dynamically build folders for each policy or to organize multiple policies into particular folders.

To configure a Manuscript to build folders dynamically or to organize policy forms into particular folders, you must:

1. Create a Manuscript field to dynamically determine the folder name and location within the forms directory.
2. Set the *outputFile* attribute of the <Manuscript.printFormsRq> (or <FormsEngine.processPrintJobRq>) request to call this field.
3. (Optional) Enable the **CreateFolder** option in the Server.config file.
4. Ensure the worker process under which the server is running has the appropriate permissions.



If you do not wish to have the system generate new subfolders automatically, you can choose not to enable this option. This will, however, impact the way forms are named and organized within your forms folders. For more information, see *Step 3: Enabling Subfolders*.

Step 1: Creating a Manuscript Field

To accommodate dynamic creation of folders for forms, you must create a Manuscript field to dynamically determine the appropriate folder path within the forms directory. For additional information, see *Managing Forms* in *EXAMPLE Forms™ Reference Guide*.

You can use a calculation to combine various components of the path and—if desired—reference additional Manuscript fields in the calculation that have their own calculations or conditional statements. Using multiple calculations allows you to build multiple levels of dynamic folders.

When creating this field, you should consider whether you want to generate a unique file name each time a form is generated. If, for example, you ran a particular quote and then amended the policy three months later, you might want the amended forms to be placed in a new folder. For this reason, you may wish to use a complex folder naming convention, such as one of the following:

- C:\Data\Policies\FORMS\Date\PolicyNumber\Time\InsuredName.pdf
- C:\Data\Policies\FORMS\PolicyNumber\Date\Time\Insured Name.pdf

Your form organization depends on how you wish to access policy forms after their creation. As a result, plan carefully before establishing your folder structure.



Avoid invalid characters such as * ? | : < > and " " when building calculations for folder paths. Using invalid characters will cause errors at runtime.

Step 2: Configuring the <ManuScript.printFormsRq> Request

To accommodate the dynamic creation of folders for forms, you must set the *outputFile* attribute on the <ManuScript.printFormsRq> (or <FormsEngine.processPrintJobRq>) request to reference the ManuScript field that calculates the correct folder path to use. To do this, modify the field that calls this request in EXAMPLE Author®.



For more information on the <ManuScript.printFormsRq> request and its attributes, see the *EXAMPLE Server® Object Reference*. For information on configuring requests in EXAMPLE Author®, see the *Guide to EXAMPLE Author®*.

Step 3: Enabling Subfolders (Optional)

When sending policy forms to a folder, if a specified folder does not exist, the system can automatically create it. To enable this feature, however, you must set the *CreateFolder* configuration setting in the Server.config file.

To set the *CreateFolder* configuration setting:

1. Open the Edit Configuration Utility.
2. In the **Virtual Directory** list, click the desired EXAMPLE Server® directory.
3. Under **Mode**, click **Form**.
4. On the **Server Config** tab, in the list of settings, click **CreateFolder**.
5. In the **value** text box, enter 1.
6. Click **Save**.
7. Click **Close**.

Once this option is enabled, if the system cannot find the appropriate folder in the path designated for a certain set of policy forms, it will create a new one.

Working with Forms in EXAMPLE Express®

In EXAMPLE Express®, users can generate and view forms using a number of EXAMPLE Forms™ features.

Generating Forms in EXAMPLE Express®

EXAMPLE Express® users can generate forms by:

- Clicking the **Create Policy Documents** hyperlink
- Using custom actions and the Policy Forms page in a product interview

Using the Create Policy Documents Link

The **Create Policy Documents** page in EXAMPLE Express® lists all print jobs available for a given policy based on the rules defined for those print jobs in EXAMPLE Author®. From this page, EXAMPLE Express® users can generate the forms in a print job as a single .pdf file. This file can then be saved or printed.



The Create Policy Documents page in EXAMPLE Express® lists all print jobs available for a given policy based on the rules defined for those print jobs in EXAMPLE Author®. When forms are generated from this page, they are not automatically attached to a policy. Instead, the user will need to manually attach them to the policy.

To generate forms from the **Create Policy Documents** page:

1. From a policy in EXAMPLE Express®, in the **Policy Actions** section in the left panel, click the **Create Policy Documents** hyperlink.
2. Click the name of the print job you wish to generate.
The forms in the print job open in Adobe Acrobat Reader as a single .pdf document.
3. (Optional) Save or print the document.

Using a Custom Action

EXAMPLE Express® users can also generate forms by clicking a custom action (defined in the product) that opens the Policy Forms page. From this page, users can:

- View all forms on a given policy and all forms available for that policy
- Select which forms to print
- Add forms to the policy
- Remove forms from the policy
- Preview forms
- Edit form data

After returning to the product interview pages, the user can click another custom action that generates the forms selected to print and attaches them to a policy, e-mails them, or saves them to a specified folder. For information on configuring these custom actions, see *Launching the Policy Forms Page* and *Configuring the <FormsEngine.processPrintJobRq>*. For information on the actions available to EXAMPLE Express® users on the Policy Forms page, see *Policy Forms Page*.

Adding/Removing Forms from a Policy

EXAMPLE Express® users can manage the forms for a particular policy from the Policy Forms page. From this page, users can:

- View all forms on a given policy and all forms available for that policy
- Select which forms to print
- Add forms to the policy

- Remove forms from the policy
- Preview forms
- Edit form data

For details about the actions available on this page, see *Policy Forms Page*.

To open the Policy Forms page:

1. In an open policy, click the **Submission** tab.
2. (Optional) If applicable, select the **Include Forms** check box.
3. Click **Policy Forms** (or any customized button that opens the Policy Forms page).

Viewing Forms in EXAMPLE Express®

You can view forms in EXAMPLE Express® from:

- **The Policy Forms page** — Allows the user to preview a form before it is attached to a policy.
- **The Create Policy Documents page** — Allows the user to generate and view all forms in a print job. This document is not automatically attached to a policy.
- **The Policy Details page** — Allows the user to view a form that is already attached to a policy.

The Policy Forms Page

To view forms from the Policy Forms page:

1. In an open policy, click the **Submission** tab.
2. (Optional) If applicable, select the **Include Forms** check box.
3. Click **Policy Forms** (or any customized button that opens the Policy Forms page).
A loading page appears while EXAMPLE Express® processes your request.
4. On the Policy Forms page, under **Actions**, click the **Preview** button. Preview the form you wish to view.
A loading page appears.
5. In the File Download dialog box, click **Open**.
The form opens in Adobe Acrobat Reader.

The Create Policy Documents Page

To view forms from the **Create Policy Documents** page:

1. From a policy in EXAMPLE Express®, in the **Policy Actions** section, click the **Create Policy Documents** hyperlink.
2. Click the name of the print job you wish to generate.
The forms in the print job open in Adobe Acrobat Reader as a single .pdf document.

The Policy Details Page

To view forms from the Policy Details page:

1. In an open policy, under **Policy Actions**, click **Policy Details**.
2. In the **Attachments** section, click the desired attachment.

3. In the File Download dialog box, click **Open**.

Appendixes

This section contains information about:

- Commercial Lines Forms
- EXAMPLE Forms™ Customization

EXAMPLE Forms™ Customization

Duck Creek Technologies, Inc. works with customers on multiple levels to customize the EXAMPLE Forms™ product. When using the EXAMPLE Forms™ product, it is important to know what role Duck Creek Technologies, Inc. will play in setting up various components of the EXAMPLE Forms™ product and in actually producing the printed forms. Standards for setting up and customizing EXAMPLE Forms™ vary widely among customers, but in general, perhaps the most clear division of customization for EXAMPLE Forms™ lies between personal lines and commercial lines forms. Duck Creek Technologies, Inc. approaches personal lines forms and commercial lines forms in very different ways.

Personal Lines

For personal lines forms, Duck Creek Technologies, Inc.'s participation in the setup and production of customer forms ranges from very little to considerable. Out of the box, the EXAMPLE Forms™ product comes with very little initial setup and customization. For personal lines customers, Duck Creek Technologies, Inc. will configure necessary forms server requests, print buttons, and the Print Job Preview screen for a customer's first line of business. Beyond this initial setup, however, customers must either customize the EXAMPLE Forms™ product themselves or commission Duck Creek Technologies, Inc. to do so for them.

When creating the actual forms, customers can either create the forms themselves or enlist company assistance. Personal lines customers who enlist the assistance of Duck Creek Technologies, Inc. must provide the company with their own source documents. These documents must be in .doc or .pdf format. Duck Creek Technologies, Inc. will then map merge fields to the appropriate fields in a ManuScript and arrange the forms into print jobs as defined by the customer.

Commercial Lines Forms

For commercial lines forms, Duck Creek Technologies, Inc. performs most forms work out of the box. Duck Creek Technologies, Inc. will work with customers to customize forms products, but all commercial lines customers receive the product with most aspects of forms production already configured.

Out-of-the-box products for commercial lines already contain the setup for forms server requests, print buttons in EXAMPLE Express®, and the Print Job Preview screen. In addition, Duck Creek Technologies, Inc. creates and includes the forms and print jobs associated with each commercial line of business. To create these forms and print jobs, Duck Creek Technologies, Inc. obtains source documents directly from ISO, NCCI, or an independent state bureau; creates merge fields for the forms; maps ManuScript fields to the merge fields; and defines the rules and print jobs associated with each form.

Commercial lines customers can either customize forms themselves or commission Duck Creek Technologies, Inc. to do so.